Interrupt

curr exec. → some exceptⁿ occurred → µp stops curr exec.

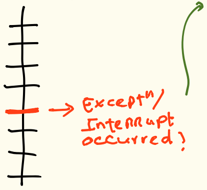
INT 21H

$\frac{5}{0}$ = undef (DIV BY ZERO)

NULL PTR EXCEPTⁿ

STACK OVERFLOW EXCEPTⁿ

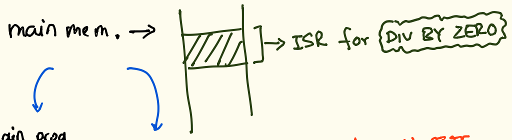
main prog

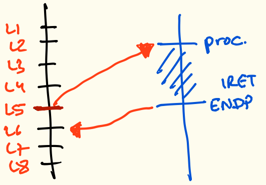

Exceptⁿ/ Interrupt occurred!

What happens when a µp is interrupted?
- ① suspends curr executⁿ
- ② jumps to ISR

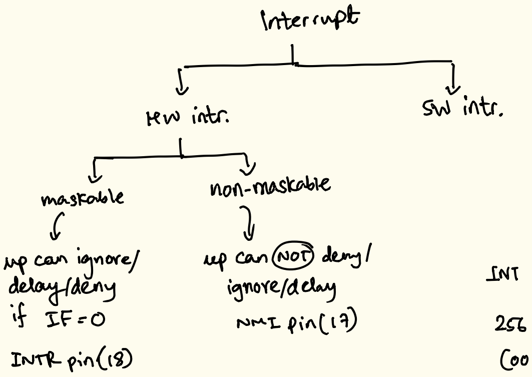
WHAT IS ISR? → other name: Interrupt Vector/ Intr. ptr/ Intr. func.

ISR → Interrupt Service Routine : special procedure/function that runs automatically when an interrupt occurs


main mem. →
ISR for DIV BY ZERO


main prog.
L1
L2
L3
L4
L5
L6
L7
L8
proc.
IRET
ENDP

L5 এ interrupt হলো
↓
ISR for that interrupt executed
↓
Returns to the next line (L6)


Interrupt
HW intr.
SW intr.
maskable
non-maskable
µp can ignore/ delay/deny if IF=0
INTR pin(18)
µp can NOT deny/ ignore/delay
NMI pin(17)

INT n ; n = interrupt type

256 types of Interrupt

(00 to FF/00 to 255)

Src	Interrupt	Type
µp	DIV Error	0
µp	Single Step	1
HW	NMI	2
µp/SW	Breakpt.	3
µp	Overflow	4
HW	INTR	Type supplied by ext. HW
SW	INT n	n(0-255)

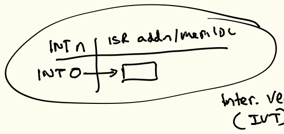
8086 intr. → 3 src.
- ① External src (INTR/NMI) → বাহির থেকে intr. দেওয়া হচ্ছে INTR/NMI pin দিয়ে
  - HW intr.
  - user-def. intr.
- ② Internal ins. INT → code এ কোনো interrupt দেওয়া থাকলে e.g. INT 21H
  - SW intr.
  - user-def. intr.
- ③ Pre-defined interrupt → যখন µp design করা হয়েছে তখন architect রা ভেবেছেন কিছু কিছু উদ্ভট intr. হতে পারে: & they have also designed solⁿs for those interrupts. e.g. DIV BY ZERO

INT 0, 1, . . . , 255

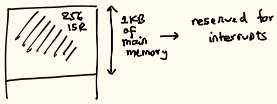
256 ISR/ Intr. vector/ Intr. ptr/ Intr. func.
↓
each ISR has an ID

INTO → DIV BY ZERO

execute ISR to handle


INT n
ISR addr/mem loc
INTO

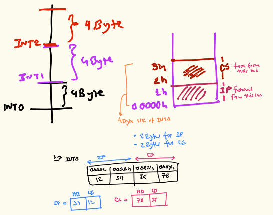
Inter. Vector Table (IVT) / Intr. Ptr. Table


256 ISR
1KB of main memory
reserved for interrupts

Tot space for interrupts = 1KB = 1024B

No. of intr = 256

$\therefore$ Space for each intr = $\frac{1024B}{256}$ = 4B.


4 Byte
INT2
4 Byte
INT1
4 Byte
INT0
3h
2h
1h
00000h
CS from from this loc
IP fetched from this loc
4 Byte ISR of INT0
• 2 Bytes for IP
• 2 Bytes for CS
e.g. INT0
IP
CS
HB
LB

00000h	00001h	00002h	00003h
12	34	56	78

IP = 34 12

CS = 78 56

$$IP_{loc} = type_d \times 4$$

$$CS_{loc} = (type_d \times 4) + 2$$

How ISR works in 8086?

1. INTR occurs
2. CPU saves curr state (Flags, CS, IP)
3. CPU jumps to ISR addr. (from IVT) → What is IVT? → other name: Intr. Ptr. Table
4. ISR executes
5. ISR ends with IRET
6. CPU returns to prev. prog.

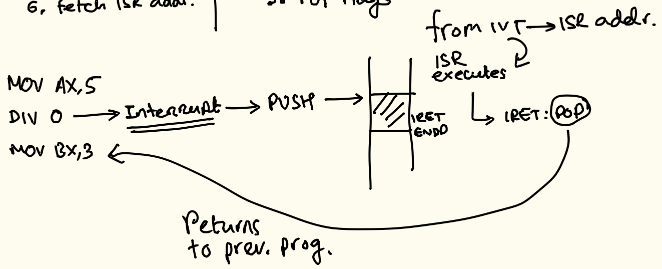
Interrupt vector Table

INT no.	ISR addr./mem loc.
INT0 →	ISR addr.
INT1 →	ISR addr.
INT2 →	ISR addr.
⋮	⋮

step ② PUSH
1. PUSH Flags
2. Clear IF
3. Clear TF
4. Push CS
5. Push IP
6. Fetch ISR addr.

IRET

step ⑤ POP
1. POP IP
2. POP CS
3. POP Flags


MOV AX,5
DIV 0
MOV BX,3
Interrupt
PUSH
from IVT → ISR addr.
ISR executes
IRET
ENDP
IRET : POP
Returns to prev. prog.

INTERRUPT PART 2 (PIC)

Interrupt generation → 2 ways : ① Polled I/O or Programmed I/O
② Interrupted I/O

① Polled I/O or Programmed I/O
- up continuously checks (polls) to see if a device is ready/not
- not ready
  - ↳ up keeps polling (busy waiting)

Disadv:
- not fast + ineff.
- time waste

② Interrupted I/O
- Device notifies up when it is ready by using an INTR signal
- handled by interrupt handler

Adv:
- efficient + faster

Interrupt Types based on ISR ID
① Predefined → 5 → Type 0: DIV ERR
1: SINGLE STEP
2: NMI
3: Breakpt
4: Overflow
⑤
② Reserved → 27
③ Available for user → 224

INT 33 instruction : SW generated interrupt → not affected by IF → non-maskable

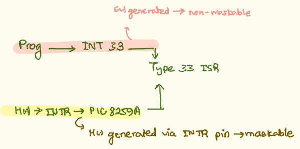
NMI/interrupt type 2 via NMI pin : non-maskable hardware BUT

INT2 vs NMI
INT 2 instruction
↳ SW generated
↓
not affected by IF
↓
non-maskable

HW interrupt via INTR : maskable by IF

INT n ≠ INTR with type n
i.e, type no. ≠ interrupt source

- Interrupt type no. এর ওপর maskable/non-maskable depend করে না
↳ Rather it depends on entry mechanism


SW generated → non-maskable
Prog → INT 33
Type 33 ISR
HW → INTR → PIC 8259A
↳ HW generated via INTR pin → maskable

"HW interrupts can also be represented as SW interrupts"
↳ A HW interrupt & a SW interrupt can use the same interrupt type no. & therefore invoke the same ISR

PIC places inten. type no. to up via data bus

up ⟷ PIC
data bus

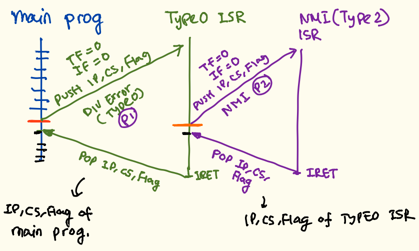
8086 interrupt priority (P) when 2/more interrupts occur at the same time?

(H)
P① Internal/SW interrupts : DIV ERR, INTn, INTO → serviced immediately + can interrupt any ISR
P② NMI (type 2) → serviced when no internal (P①) interrupt occurs + can interrupt any ISR (including an ISR servicing a higher-priority internal/SW interrupt)
P③ INTR → if IF = 1 + no NMI occurs
↳ not possible to attempt if IF = 0 & TF = 0
P④ Single Step/Trap → serviced at last
↳ not possible if IF = TF = 0
(L)
priority

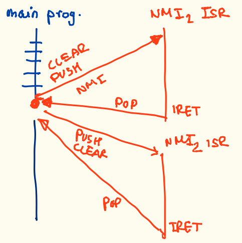
e.g
DIV
↓
Divide Error ISR starts
↓
NMI interrupts Divide Error ISR
↓
NMI ISR
↓
IRET
↓
Continue Divide Error ISR
↓
IRET
↓
Main Program

- NMI is not ignored simply bcz an internal interrupt is occurring
- Internal interrupt gets 1st entry but NMI can interrupt its ISR
  - ↳ i.e., an interrupt procedure always may be interrupted by a request arriving on NMI.

Example 1: main prog → SW (P1) inter. → NMI (P2) inter.
↳ INTn + NMI (diff" priority)


main prog
Type0 ISR
NMI (Type 2) ISR
TF=0
IF=0
PUSH IP, CS, Flag
DIV Error (Type0)
P1
TF=0
IF=0
PUSH IP, CS, Flag
NMI
P2
Pop IP, CS, Flag
IRET
Pop IP, CS, Flag
IRET
IP, CS, Flag of main prog.
IP, CS, Flag of TYPE0 ISR

Example 2: main prog → NMI (P2) inter. → NMI (P2) inter.
↳ NMI + NMI same priority


main prog.
NMI2 ISR
CLEAR
PUSH
NMI
POP
IRET
PUSH
CLEAR
NMI2 ISR
POP
IRET

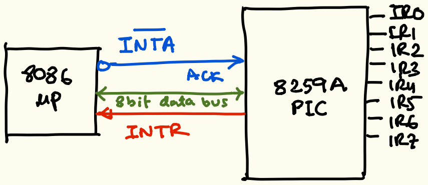
Example 3: main prog → INTR → INTR
↳ INTR + INTR (Same priority)
→ Programmable Interrupt Controller (PIC)

Task: ① prioritization

② provides IVT no. to µp

PROGRAMMABLE INTERRUPT CONTROLLER (PIC)

↳ works as an assistant for µp to deal w INTR



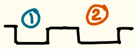
NOTE: 8259A deals with maskable interrupts (INTR)

NOT non-maskable interrupts (NMI)

① 8259A informs µp 8086 about maskable interrupt via INTR

② If µp wants to handle, 8086 sends ACK to 8259A via $\overline{\text{INTA}}$

2 low signals



① Ready to serve the interrupt ② give µp the interrupt no./ISR no./type

③ Data bus 8-bit → WHY? ↷

Total interrupt type = 256 = 2^{8} → so 8 bits to represent/256 types of interrupts address

Formula: $\log_2(n)$ bit data bus for n type of interrupts

If 4096 types → then $\log_2(4096)$ = 12 bits data bus to be used.

8086 sets priority on IR0-7 in 2 ways:

default mode

① Fixed Priority Mode : IR0 → highest priority ↓
IR7 → lowest priority

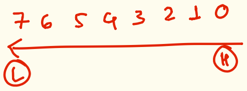
Say, µp serving IR4 now. At that time, INTR from IR1 and IR7 came concurrently.

µp will prioritize IR0.

② Rotating priority mode: priority given sequentially

Say, µp serving INTR from IR5. At that time, INTR via IR0 came. But µp will give priority to IR6, IR7, IR0, . . . in that order.

8259A architecture



- has 3 registers
  - ① IRR (Interrupt Req. Reg.) : tracks pins where interrupt has occurred/been triggered
  - ② ISR (Interrupt Service Reg.): tracks which INTR is being serviced by the µp now
  - ③ IMR (Interrupt Mask Reg.) : tracks which INTR are masked by the µp

all 8bit reg.
↓
SAME size

NOTE: Masking done by µp NOT by the PIC

- Priority Resolver (PR) → Decides µp কোন INTR কে serve করবে

④ Cascading 8259A PIC

master PIC
slave PIC

CAS0, 1, 2 → used to identify which Slave 8259A should respond to an interrupt acknowledge →

3 pins
as 8 slaves can be connected via IRQ-7

$\log_2(8) = 3$ pins

$\lceil \log_2(n) \rceil$ pins

When a slave requests an interrupt:

1. Slave raises INTR req via master's IR lines
2. Master recognizes the interrupt
3. During $\overline{\text{INTA}}$ (ACK), master puts a 3-bit code on CAS0-2 to select the slave whose req. will be serviced
4. The slave places the appropriate inter. type onto the data bus

A fully cascaded 8259A sys. can have : 1 master + $\overset{\text{max}}{8}$ slaves = 9 PICs
+
can provide 8 × 8 = 64 interrupt req. inputs

64 int. req
↓
8 + 7N
= 8 + 7(8)
↓
why (n-1)? ; $n = \overset{\text{IR}}{\text{pins}}$
→ bcz every slave needs to occupy one IR-line of master
↓
Every slave adds n inter. srcs but consumes 1 IR-line of the master

If each PIC has n IR-lines & N slaves,
$\text{Total}_{\text{req}} = n + (n-1)N$